

Appendix

0.1 Use of Large Language Models

We thank large language models for polishing the manuscript’s language, suggesting editorial revisions, and assisting with coding. All outputs are reviewed, verified, and integrated by the authors, who take full responsibility for the content and any remaining errors.

0.2 Implementation Details

Training Hyperparameters We train our VITA-E model in two stages. First, we finetune the VITA-1.5 model on the embodied scenario vision-language data using the DeepSpeed with ZeRO-3 configuration. Then, we follow GR00T to train our model on the Libero simulation environment or the collected real-robot data. In terms of the Libero simulation environment, we first pretrain it on Libero-90, and then finetune it on the mixture of four task suites of Libero-10. The hyperparameters we used to train our VITA-E model are listed in Table 1.

Table 1: VITA-E training hyperparameters.

Hyperparameters	Value
batch size	64
gradient accumulation steps	1
learning rate	1e-4
optimizer	AdamW
learning rate schedule	cosine decay
warmup ratio	0.05
training steps	20000

Table 2: VITA-E model hyperparameters.

Hyperparameters	Libero	Real Robot
top image	224×224	224×224
wrist image	224×224	-
input state dim	7	26
output action dim	7	26
history length	1	1
future action prediction	16	16
tune visual	False	False
tune LLM	False	False
tune diffusion	True	False
tune projector	True	True

Model Hyperparameters We listed the key parameters in our VITA-E model design in Table 2. Most model hyperparameters follow those in GR00T to ensure a fair comparison.

0.3 Voice Response Speed Comparison

To further demonstrate the advantages of VITA-E in low voice response latency, we compared VITA-E against a mainstream cascaded pipeline with similar parameter counts on H100 GPU: Whisper Small [2] + VITA-1.5 + CosyVoice-300M [1]. In this pipeline, user speech is first transcribed into text by the Whisper model; then, the transcribed text and images are input into the VITA-1.5 model to generate response text; finally, the response text is converted into speech by the CosyVoice model. The results of this comparative experiment are shown in Table 3:

Table 3: Voice response latency comparison.

Module	VITA-E (End-to-End)	Pipeline (Cascaded)
ASR	-	0.061s
VLM Inference	0.740s	0.330s
TTS Generation	1.444s	3.172s
Total Latency	2.184s	3.563s

Table 4: Details of VITA-1.5 VLM inference token count.

Model	Input	Image	System Prompt	Avg. Input Audio/Text Tokens	Avg. Output Tokens
VITA-E	Audio+Image	256	151	56	61.6
Pipeline	Text+Image	256	151	13.0	20.0

Since the input to VITA-1.5 in the pipeline is text generated by ASR, whereas the input to VITA-1.5 in VITA-E is speech, there are discrepancies in the number of input and output tokens. The specific details are shown in Table 4.

The results indicate that the inference speed of the end-to-end VITA-1.5 model used in VITA-E is comparable to that of the Whisper + VITA-1.5 combination with similar parameters (when normalized for token count). However, the TTS generation speed is significantly faster than CosyVoice, which adopts the FlowMatching architecture. Therefore, our VITA-1.5 model achieves faster response speeds than the baseline pipeline using similar parameters.

0.4 Prompts

In this section, we present the prompt used to generate synthetic vision-language data for fine-tuning the VLM model as detailed in Table 5 to 8. We synthesize four categories of instruction-answer pairs to simulate the robot’s responses to various user instructions, including: performing an action, being unable to complete an instruction,

emergency stop, and task completed. After generating the data, we manually insert special tokens at the required locations.

<Image>

Please act as the robotic arm shown in the image. There are several objects on the table in front of you. Your task is to generate 3 different operation instructions based on these objects, and provide a corresponding robot response for each instruction.

Before giving the instructions, analyze the attributes, positions, colors, and shapes of the manipulable objects to describe and locate them more precisely. However, do not output the analysis process.

Instructions should sound natural and appropriate, and the operations must comply with the physical properties and spatial relationships of the objects.

If the instruction is unambiguous, keep it as concise as possible by omitting unnecessary details such as color, material, or relative position, for example: "Pick up the bottle", "Open the drawer", or "Place the plate on the left side of the cabinet". Avoid using vague descriptions such as "in the middle/center of the table", "near", "beside", or "next to", as these could apply to many objects. Instead, use precise relative positioning, such as "to the left front of an object", "on top of an object", "between object A and object B", "to the right back of an object", or "behind an object".

If the instruction is ambiguous, describe the object as precisely as possible, for example: "Pick up the black bottle to the right of the plate", or "Take the apple from the plate and place it on the cabinet to the right". Similarly, avoid vague descriptions like "pick up the thing on the table".

After each instruction, provide a more specific robot response. The response can be as varied and personal as possible. The response could start with a human-like phrase such as "I will pick up", "I will take", "I will help you", or "I will close", and then clearly state the object name, possibly including additional spatial details to help locate it.

Your task:

Generate 3 different operation instructions and corresponding robot responses. Instructions can involve a single object, such as "Pick up the cola", or a combination of multiple objects, such as "Pick up the apple from the table and put it on the plate". Please ensure that the objects involved actually exist in the image and that the operations are physically feasible.

For each task, please follow the format below, and output the content of the Instruction and the Response in Chinese:

Start Task <task id>

Instruction: ...

Response: ...

End Task <task id>

Table 5: Prompt for constructing action instructions and robot responses data.

<Image>

Please act as the robotic arm shown in the image. There are several objects placed on the table in front of you. Your task is to generate 3 different operation instructions that the robot will refuse to execute based on these objects, and provide a corresponding robot response for each instruction.

Before giving the instructions, analyze the attributes, positions, colors, and shapes of the manipulable objects to describe and locate them more precisely. However, do not output the analysis process.

Each instruction should either involve an object not present in the image or describe an action that is physically impossible, so the robot cannot execute it.

If the instruction is unambiguous, keep it as concise as possible by omitting unnecessary details such as color, material, or relative position, for example: "Pick up the bottle", "Open the drawer", or "Place the plate on the left side of the cabinet." Avoid using vague descriptions such as "in the middle/center of the table", "near", "beside", or "next to", as these could apply to many objects. Instead, use precise relative positioning, such as "to the left front of an object", "on top of an object", "between object A and object B", "to the right back of an object", or "behind an object".

If the instruction is ambiguous, describe the object as precisely as possible, for example: "Pick up the black bottle to the right of the plate", or "Take the apple from the plate and place it on the cabinet to the right". Similarly, avoid vague descriptions like "pick up the thing on the table".

After each instruction, provide a more specific robot response. The response can be as varied and personal as possible. The response can start with a human-like phrase, such as "I don't see...", "Sorry, ... does not exist", "I don't see...", "There is no ... on the table", "I can't...", "... cannot be...", etc., clearly stating the object name, and could include some spatial details.

Your task:

Generate 3 different operation instructions that the robot will refuse to execute and the corresponding robot responses. Instructions can involve a single object, such as "Pick up the cola", or a combination of multiple objects, such as "Pick up the apple from the table and put it on the plate". Please ensure that at least one of the objects involved in the instruction does not exist in the image, or the operation is physically impossible.

For each task, please follow the format below, and output the content of the Instruction and the Response in Chinese:

Start Task <task id>

Instruction: ...

Response: ...

End Task <task id>

Table 6: Prompt for constructing unfulfillable action instructions and robot responses data.

<Image> Please act as the robotic arm shown in the image. You are currently performing an operational task. Generate new instructions to interrupt the ongoing task. The instructions should be as diverse and concise as possible, such as “Stop”, “Terminate”, etc. After each instruction, provide a more specific robotic response. The responses should also be as varied and personalized as possible, such as “Understood, I will end the current operation”, “I will immediately pause the task”, or “Received, aborting the current process”, etc. Your task: Generate 3 different instructions to interrupt the robot’s operation, along with corresponding robotic responses. The instructions and responses should be natural and suitable for a real robot-human interaction. For each task, please follow the format below, and output the content of the Instruction and the Response in Chinese: Start Task <task id> Instruction: ... Response: ... End Task <task id>

Table 7: Prompt for constructing emergency stop instructions and robot responses data.

<Image>

Please act as the robotic arm shown in the image. You have already completed an operation instruction. The image shows the scene after the operation instruction is completed. Please infer what instruction you have completed.

Before giving the instructions, analyze the attributes, positions, colors, and shapes of the manipulable objects to describe and locate them more precisely. However, do not output the analysis process.

Instructions should sound natural and appropriate, and the operations must comply with the physical properties and spatial relationships of the objects.

If the instruction is unambiguous, keep it as concise as possible by omitting unnecessary details such as color, material, or relative position, for example: "Pick up the bottle", "Open the drawer", or "Place the plate on the left side of the cabinet". Avoid using vague descriptions such as "in the middle/center of the table", "near", "beside", or "next to", as these could apply to many objects. Instead, use precise relative positioning, such as "to the left front of an object", "on top of an object", "between object A and object B", "to the right back of an object", or "behind an object".

If the instruction is ambiguous, describe the object as precisely as possible, for example: "Pick up the black bottle to the right of the plate", or "Take the apple from the plate and place it on the cabinet to the right". Similarly, avoid vague descriptions like "pick up the thing on the table".

Your task:

Generate one instruction that has already been completed. Instructions can involve a single object, such as "Pick up the cola", or a combination of multiple objects, such as "Pick up the apple from the table and put it on the plate". Ensure that the objects involved truly exist in the image and that the operation has been completed.

For each task, please follow the format below, and output the content of the Instruction in Chinese:

Start Task <task id>

Instruction: ...

End Task <task id>

Table 8: Prompt for constructing action instructions data that has been completed.

References

- [1] Zhihao Du, Qian Chen, Shiliang Zhang, Kai Hu, Heng Lu, Yexin Yang, Hangrui Hu, Siqi Zheng, Yue Gu, Ziyang Ma, Zhifu Gao, and Zhijie Yan. Cosyvoice: A scalable multilingual zero-shot text-to-speech synthesizer based on supervised semantic tokens. *ArXiv*, abs/2407.05407, 2024.
- [2] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, C. McLeavey, and I. Sutskever. Robust speech recognition via large-scale weak supervision. pp. 28492–28518, 2022.